



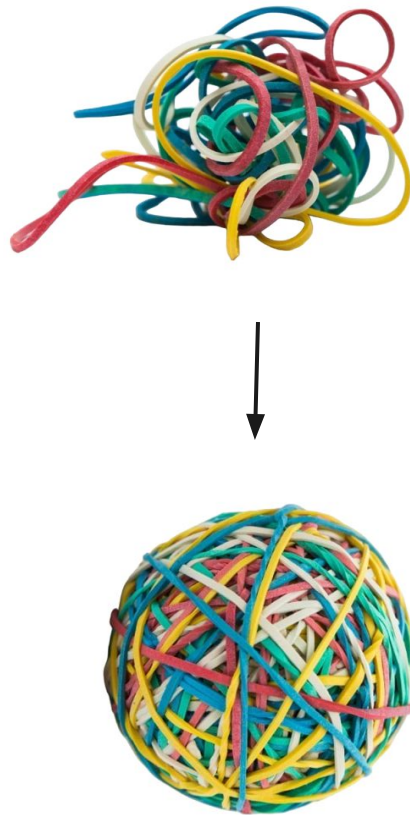
Code Refactoring

~ The art of improving the design of existing code

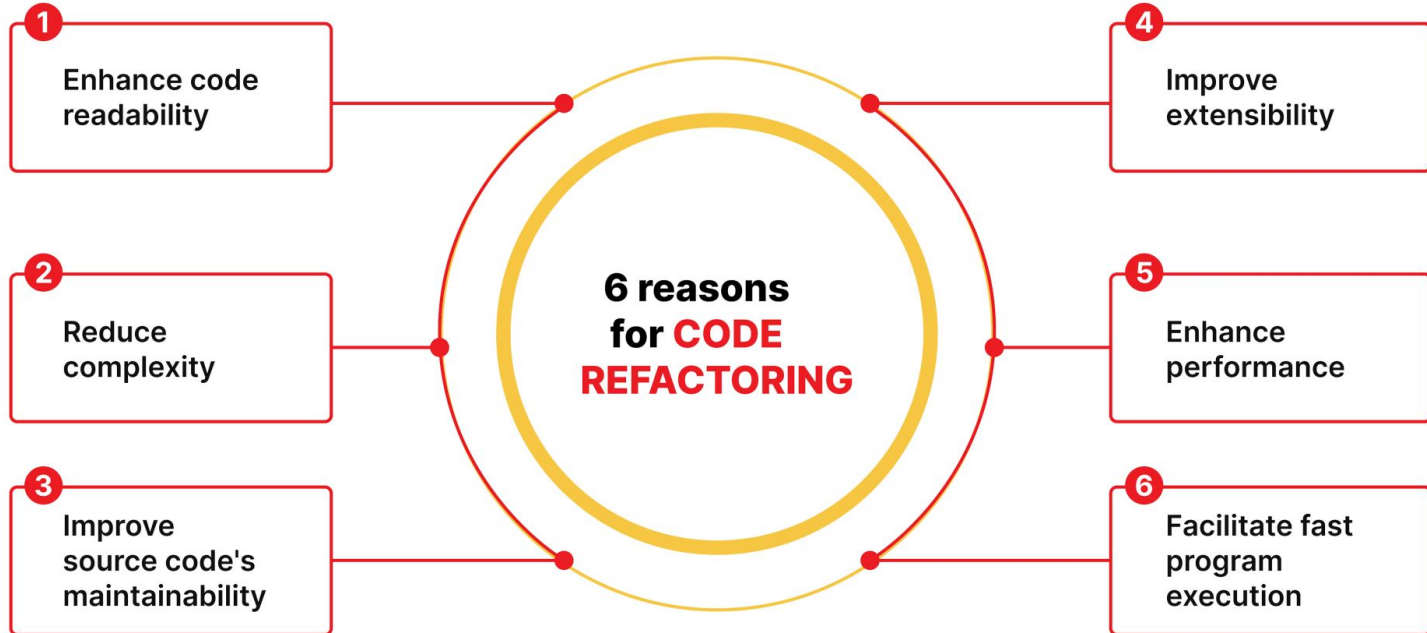
COMS 4156

What is Refactoring?

- **Definition:** Refactoring is the process of **restructuring code without changing its external functionality** or behavior.
- **Objective:** To restructure existing code, making it more **readable, maintainable, and extendable** without altering its external behavior.
- **Types of Refactoring:**
 - **Renaming** (e.g., changing ambiguous variable names).
 - **Extracting** (e.g., moving repeated code into functions).
 - **Reorganizing** (e.g., breaking large functions into smaller, focused ones).



Why Refactoring?





Manual Refactoring

#Code Sample

```
struct Item {
    double price;
    int quantity;
};

int main() {
    std::vector<Item> order = {[0]={.price=20.0, .quantity=2}, [1]={.price=15.0, .quantity=1}, [...]{30.0, 3}};
    double total = 0.0;
    double with_discount = 0.0;
    double with_tax = 0.0;
    double with_shipping = 0.0;

    for (Item i : order) {
        total += i.price * i.quantity;
    }

    if (total > 100) {
        with_discount = total - (total * 0.05);
    } else {
        with_discount = total;
    }

    with_tax = with_discount + (with_discount * 0.10);

    if (with_tax < 50) {
        with_shipping = with_tax + 5;
    } else {
        with_shipping = with_tax;
    }

    std::cout << "Total Price: $" << with_shipping << std::endl;
    return 0;
}
```

Check this code.
Is it a good code?

Does the code work?

-> **Yes... it works**

Is it a good code?

-> **NO!**



#Code Sample

Issues with the previous code function :



- **Naming of Variables:** Variable names, like `with_discount`, etc., could be more descriptive.
- **Long, Complex Methods:** The main function contains all the logic, making it hard to read and reuse. The logic could be extracted into separate methods.
- **Duplication of code:** The calculation logic for the total price (discount, tax, shipping) is repeated in different places and could be consolidated.
- **No Use of Constants:** Magic numbers are used without context.

#Code Sample

```
struct Item {
    double price;
    int quantity;
};

int main() {
    std::vector<Item> itemsInCart = {{20.0, 2}, {15.0, 1}, {30.0, 3}};
    double subtotal = 0.0;
    double discountTotal = 0.0;
    double totalWithTax = 0.0;
    double finalTotal = 0.0;

    for (Item item : itemsInCart) {
        subtotal += item.price * item.quantity;
    }

    if (subtotal > 100) {
        discountTotal = subtotal - (subtotal * 0.05);
    } else {
        discountTotal = subtotal;
    }

    totalWithTax = discountTotal + (discountTotal * 0.10);

    if (totalWithTax < 50) {
        finalTotal = totalWithTax + 5;
    } else {
        finalTotal = totalWithTax;
    }

    std::cout << "Total Price: $" << finalTotal << std::endl;
```

Refactoring Step #1 -

Rename variables with descriptive names to make the purpose clearer, indicating what they represent.

This improves readability.

```

double calculateSubtotal(const std::vector<Item>& items) {
    double subtotal = 0.0;
    for (const Item& item : items) {
        subtotal += item.price * item.quantity;
    }
    return subtotal;
}

double applyDiscount(double subtotal) {
    const double DISCOUNT_RATE = 0.05;
    return subtotal > 100 ? subtotal - (subtotal * DISCOUNT_RATE) : subtotal;
}

double applyTax(double totalAfterDiscount) {
    const double TAX_RATE = 0.10;
    return totalAfterDiscount + (totalAfterDiscount * TAX_RATE);
}

double applyShipping(double totalAfterTax) {
    const double SHIPPING_FEE = 5.0;
    return totalAfterTax < 50 ? totalAfterTax + SHIPPING_FEE : totalAfterTax;
}

int main() {
    std::vector<Item> itemsInCart = {{20.0, 2}, {15.0, 1}, {30.0, 3}};

    double subtotal = calculateSubtotal(itemsInCart);
    double discountTotal = applyDiscount(subtotal);
    double totalWithTax = applyTax(discountTotal);
    double finalTotal = applyShipping(totalWithTax);

    std::cout << "Total Price: $" << finalTotal << std::endl;
    return 0;
}

```

Refactoring Step #2 -

Extracting separate methods to improve readability and reusability. This makes the main function shorter and more readable.

Also replace the hard-coded values (e.g., 5 for the shipping fee and 0.05 for the discount rate) with constants to improve readability and reduce error risk if these values change.

```

double calculateSubtotal(const std::vector<Item>& items) {
    double subtotal = 0.0;
    for (const Item& item : items) {
        subtotal += item.price * item.quantity;
    }
    return subtotal;
}

double applyShipping(double totalAfterTax) {
    const double SHIPPING_FEE = 5.0;
    return totalAfterTax < 50 ? totalAfterTax + SHIPPING_FEE : totalAfterTax;
}

double applyTotalAdjustments(double subtotal){
    const double DISCOUNT_RATE = 0.05;
    const double TAX_RATE = 0.10;
    const double DISCOUNT_THRESHOLD = 100;

    double totalAfterDiscount = (subtotal > DISCOUNT_THRESHOLD)
        ? subtotal * (1 - DISCOUNT_RATE)
        : subtotal;

    double discountedTotalAfterTax = totalAfterDiscount + (totalAfterDiscount * TAX_RATE);
    double finalTotalAfterShipping = applyShipping(discountedTotalAfterTax);

    return finalTotalAfterShipping;
}

int main() {
    std::vector<Item> itemsInCart = {{20.0, 2}, {15.0, 1}, {30.0, 3}};

    double subtotal = calculateSubtotal(itemsInCart);
    double finalTotal = applyTotalAdjustments(subtotal);

    std::cout << "Total Price: $" << finalTotal << std::endl;
    return 0;
}

```

Refactoring Step #3 -

Simplifying the logic of the methods to make the functions more concise.

This also minimizes the logic in main by calculating the total in a single streamlined function.



Automatic Refactoring

Refactoring with a tool

Potential Refactoring with tools:



- Renaming Variables
- Extracting Methods (e.g., extracting the discount calculation logic).
- Inlining Variables (e.g., simplifying expressions)
- Organizing Imports (if relevant)



Small Demo

Refactoring tools support

Different IDEs and plugins for different languages :



- **IntelliJ IDEA** (for Java, Kotlin, and other languages): It has excellent refactoring tools built in, such as “Rename,” “Extract Method,” “Inline Method,” etc.
- **PyCharm** (for Python): Similar to IntelliJ, it supports refactoring tools like “Rename,” “Extract Method,” “Move,” and “Change Signature.”
- **CLion** (for C++): Built-in Clang-Tidy integration helps with additional refactoring, including simplifying code expressions and modernizing syntax.
- **Visual Studio Code** (with extensions): VS Code can integrate with extensions like “Refactorix” and “clangd” for refactoring.
- **Rubocop and RubyMine** (for Ruby): These tools support various automated refactorings for Ruby code.

Wrap Up



- The speed and accuracy provided by refactoring tools.
- The manual effort required for deeper understanding and architectural changes.
- A combination of both approaches ensures optimal code quality, readability, and maintainability.



Questions?